

Guía de Buenas Prácticas de Codificación en Diseño Electrónico

01/08/2025 (V.1)

Carlos Cruz de la Torre

Departamento de Electrónica

Universidad de Alcalá

Mail: carlos.cruzt@uah.es

Web: www.carloscruztorre.com

Web: <https://www.uah.es/es/estudios/profesor/Carlos-Cruz-de-la-Torre/>

Guía de Buenas Prácticas VHDL

Objetivo: Establecer una guía de recomendaciones generales para mejorar legibilidad, mantenibilidad y síntesis fiable.

1) Nombres, archivos y jerarquía

R01. Usar nombres significativos para bloques, señales y archivos.

```
emergencia.vhd
emergencia_tb.vhd
uart_rx.vhd
```

R02. Prefijos de puertos: entradas i_*, salidas o_*. Si se usan nombres del fichero de constraints, deben coincidir exactamente.

```
entity contador is
  port (
    i_clk   : in  std_logic;           -- reloj principal
    i_rstn  : in  std_logic;           -- reset activo bajo
    o_done  : out std_logic            -- fin de conteo
  );
end entity;
```

R03. Sufijo _tb para los bancos de pruebas (testbench).

R04. Toda entidad de diseño se instancia en su testbench y en la jerarquía HW.

```
-- Testbench
uut: entity work.contador(rtl)
  port map (
    i_clk  => clk,
    i_rstn => rstn,
    o_done => done
  );
```

R05. Preferir jerarquía por instanciación antes que esquemas gráficos (mejor control de versiones).

R06. Asociación de puertos explícita por NOMBRE (evitar posicional).

```
uart_i: entity work.uart_rx
  port map (
    i_clk  => i_clk,
    i_rstn => i_rstn,
    i_rxd  => i_rxd,
    o_data => rx_data,
    o_vld  => rx_vld
  );
```

2) Señales, variables y tipos

R07. Evitar inicializar señales en la declaración en lógica sintetizable. Usa reset dentro del proceso.

```
-- Evitar: signal foo : std_logic := '1'; -- en RTL
process(i_clk) is
begin
    if rising_edge(i_clk) then
        if i_rstn='0' then
            foo <= '0';
        else
            foo <= next_foo;
        end if;
    end if;
end process;
```

R08. Variables ≠ señales. Las variables se actualizan inmediatamente; las señales al final del delta. Úsalas para cálculos locales, no para comunicar procesos.

```
process(a)
    variable acc : unsigned(3 downto 0);
begin
    acc := (others => '0');
    for i in a'range loop
        if a(i)='1' then acc := acc + 1; end if;
    end loop;
    b <= std_logic_vector(acc);
end process;
```

R09. Puertos en std_logic/std_logic_vector; aritmética interna con signed/unsigned. Convertir en los límites del módulo.

R10. Tipos adecuados: std_logic para señales individuales, unsigned para contadores/direcciones y signed para magnitudes con signo.

```
use ieee.numeric_std.all;
signal addr : unsigned(15 downto 0);
signal temp : signed(15 downto 0);
-- Límite de módulo
o_sum <= std_logic_vector(sum_u);
```

3) Procesos y estilo de código

R11. Usar procesos con if/case para lógica combinacional y secuencial; equivalen a when-else/with-select pero son más escalables.

```
-- Combinacional con CASE
process(sel, a, b, c)
begin
    case sel is
        when "00"    => y <= a;
        when "01"    => y <= b;
```

```

        when others => y <= c;
    end case;
end process;

```

```

-- Concurrente simple (válido para lógica trivial)
y_or <= a or b;

```

R12. Usar rising_edge(clk) o falling_edge(clk) en lugar de clk'event and clk='1'.

```

process(i_clk) is
begin
    if rising_edge(i_clk) then
        -- ...
    end if;
end process;

```

R13. Comparaciones eficientes: usar < 0 o >= 0 para signed (solo comprueban el MSB).

```

if signed(temp) < 0 then
    neg <= '1';
else
    neg <= '0';
end if;

```

4) Máquinas de estado (FSM)

R14. Separar Registro / Próximo Estado / Salida (patrón Mealy).

```

type state_t is (IDLE, BUSY);
signal ps, ns : state_t;

-- Registro de estado
process(i_clk) is
begin
    if rising_edge(i_clk) then
        if i_rstn='0' then ps <= IDLE; else ps <= ns; end if;
    end if;
end process;

-- Próximo estado
process(ps, start, done) is
begin
    ns := ps;
    case ps is
        when IDLE => if start='1' then ns := BUSY; end if;
        when BUSY => if done='1' then ns := IDLE; end if;
    end case;
end process;

-- Salida
o_busy <= '1' when ps=BUSY else '0';

```

5) Organización de archivos y legibilidad

R15. Una entidad por archivo (salvo configuraciones).

R16. No separar entidad y arquitectura en archivos distintos. Si hay varias arquitecturas, comparten la misma entidad.

R17. Una entidad = una responsabilidad. Evita mezclar funcionalidades independientes (salvo entidades estructurales).

R18. Un puerto por línea y con comentario.

```
port (  
    i_clk   : in   std_logic;           -- reloj 100 MHz  
    i_rstn  : in   std_logic;           -- reset activo bajo  
    i_d     : in   std_logic_vector(7 downto 0); -- datos  
    o_q     : out  std_logic_vector(7 downto 0) -- salida registrada  
);
```

R19. Evitar acentos y caracteres no ASCII en identificadores (mejor inglés).

R20. Una instrucción ejecutable por línea (mejores diffs).

R21. Comentarios que aporten valor (el porqué). Añade cabecera breve al archivo.

```
--! @brief Contador módulo N con enable y reset síncrono  
--! @author Equipo HW  
--! @date 2025-08-17
```

R22. No mantener cementerios de código comentado. Usa control de versiones para históricos.

R23. Evitar nombres de 1 letra (i, n, ...). Prefiere byte_cnt, addr_max, etc.

R24. Usar sufijo _t para tipos y subtipos.

```
subtype byte_t is std_logic_vector(7 downto 0);  
type state_t is (IDLE, BUSY, ERR);
```

R25. Convención de mayúsculas/minúsculas consistente (p.ej., lower_snake_case o lowerCamelCase).

R26. Nombres de arquitectura convencionales: rtl, behavioral, structural, tb.

R27. Señales activas bajas con una sola notación (recomendado sufijo _n).

```
if i_rstn='0' then  
    -- reset  
end if;
```

R28. Alias interno para salidas: permite leer la salida dentro de la arquitectura sin retroalimentar el puerto.

```
signal out_i : std_logic_vector(7 downto 0);  
o_out <= out_i;           -- puerto  
-- uso interno  
out_i <= a and b;
```

R29. Nombrar claramente relojes y resets: clk, clk_100m, rst_n, rst_clk50m_n.

6) Mini-ejemplo integrador

Temporizador con enable y reset síncrono aplicando varias reglas a la vez:

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity timer is
  generic ( G_MAX : unsigned(15 downto 0) := to_unsigned(9999,16) );
  port (
    i_clk      : in  std_logic;      -- 100 MHz
    i_rstn     : in  std_logic;      -- reset activo bajo
    i_start    : in  std_logic;      -- arranque
    o_done     : out std_logic        -- pulso de fin
  );
end entity;

architecture rtl of timer is
  signal cnt      : unsigned(15 downto 0);
  signal done_i   : std_logic;
begin
  o_done <= done_i;

  process(i_clk)
  begin
    if rising_edge(i_clk) then
      if i_rstn='0' then
        cnt      <= (others=>'0');
        done_i <= '0';
      else
        if i_start='1' then
          if cnt = G_MAX then
            cnt      <= (others=>'0');
            done_i <= '1';
          else
            cnt      <= cnt + 1;
            done_i <= '0';
          end if;
        else
          cnt      <= (others=>'0');
          done_i <= '0';
        end if;
      end if;
    end if;
  end process;
end architecture;
```

Errores Generales a evitar en VHDL

Objetivo: Definir los errores frecuentes en diseño electrónico con VHDL y mostrar cómo evitarlos con ejemplos prácticos (mal/bien).

G01. Usar tipos estándar IEEE y adecuados al propósito — Categoría: Error

Evita librerías no estándar y hacer aritmética con `std_logic_vector`. Para contadores y direcciones usa `unsigned`; para magnitudes con signo, `signed`.

```
-- MAL: librería no estándar y aritmética sobre std_logic_vector
```

```

use ieee.std_logic_unsigned.all;
signal cnt : std_logic_vector(7 downto 0);
cnt <= cnt + 1;

```

```

-- BIEN: numeric_std con tipos adecuados
use ieee.numeric_std.all;
signal cnt_u : unsigned(7 downto 0);
cnt_u <= cnt_u + 1;

```

G02. Evitar asignaciones múltiples a la misma señal en la misma región de código —
Categoría: Warning

Múltiples drivers generan conflictos. Centraliza la lógica en un único proceso o usa señales intermedias.

```

-- MAL: dos procesos escriben 'q_i'
process(clk) begin if rising_edge(clk) then if set='1' then q_i <= '1';
end if; end if; end process;
process(clk) begin if rising_edge(clk) then if rst='1' then q_i <= '0';
end if; end if; end process;

```

```

-- BIEN: un solo proceso resuelve todas las condiciones
process(clk) begin
  if rising_edge(clk) then
    if rst='1' then q_i <= '0';
    elsif set='1' then q_i <= '1';
    else q_i <= q_i;
    end if;
  end if;
end process;

```

G03. Evitar constantes de tamaño prefijado cuando afecten a portabilidad — Categoría:
Warning

Parametriza con genéricos o usa atributos 'length' y subtipos para desacoplar el ancho.

```

-- MAL: literal con tamaño/encoding fijo
signal cnt : unsigned(15 downto 0);
if cnt = x"03E8" then ... end if; -- 1000 hardcoded a 16 bits

-- BIEN: constante dependiente del ancho
constant C_MAX : unsigned(cnt'range) := to_unsigned(1000, cnt'length);
if cnt = C_MAX then ... end if;

```

G04. Evitar asignar vectores de reset con cadenas de bits fijas — Categoría: Nota

Usa (others=>'0'/'1') o atributos de tamaño; evita errores al cambiar anchos.

```

-- MAL
shift_reg <= "00000000";

-- BIEN
shift_reg <= (others => '0');

```


G05. Estilo consistente de FSM (tipos enumerados y dos/tres procesos) — Categoría: Error

Modela estados con un tipo enumerado y permite que la herramienta elija la codificación.
Evita codificar estados como `std_logic_vector`, que limita cambios.

```
-- MAL: estados codificados en std_logic_vector
signal st : std_logic_vector(1 downto 0);
-- when "00" => ...

-- BIEN: tipo enumerado
type state_t is (IDLE, READ, WRITE);
signal ps, ns : state_t;
-- registro de estado
process(clk) begin
    if rising_edge(clk) then
        if rst='1' then ps <= IDLE; else ps <= ns; end if;
    end if;
end process;
-- próximo estado
process(ps, start, done) begin
    ns <= ps;
    case ps is
        when IDLE => if start='1' then ns <= READ; end if;
        when READ => if done='1' then ns <= WRITE; end if;
        when WRITE=> ns <= IDLE;
    end case;
end process;
```

G06. Transiciones seguras en FSM — Categoría: Error

Debes definir un estado de reset, cubrir todos los casos y evitar estados inalcanzables.
Incluye paths por defecto controlados (no 'latches').

```
-- MAL: CASE incompleto y sin reset
process(ps, a) begin
    case ps is
        when IDLE => if a='1' then ns <= BUSY; end if;
        -- falta when others
    end case;
end process;

-- BIEN: reset y cobertura completa
process(ps, a) begin
    ns <= ps; -- por defecto
    case ps is
        when IDLE => if a='1' then ns <= BUSY; end if;
        when BUSY => ns <= IDLE;
        when others => ns <= IDLE;
    end case;
end process;
```

G07. Evitar rangos desajustados (ancho y orientación) — Categoría: Warning

Los anchos y direcciones deben coincidir. Si cambias orientación, reslice explícitamente.

```
-- MAL: anchos u orientación incompatibles
signal a : std_logic_vector(0 to 7);
signal b : std_logic_vector(7 downto 0);
b <= a; -- desajuste de orientación
-- BIEN: reslicing explícito
b <= a(7 downto 0); -- invierte el slice
-- o unifica la declaración para ambos
```

G08. Listas de sensibilidad completas en procesos combinacionales — Categoría: Warning

Incluye todas las señales leídas o usa VHDL-2008 'process(all)' para evitar latches/discordancias.

```
-- MAL
process(a) begin
    y <= a and b; -- falta b en la sensibilidad
end process;
-- BIEN (VHDL-93)
process(a, b) begin
    y <= a and b;
end process;

-- BIEN (VHDL-2008)
process(all) begin
    y <= a and b;
end process;
```

G09. Subprogramas bien estructurados (sin recursión ni efectos laterales) — Categoría: Warning

Usa funciones/procedimientos con un único punto de retorno claro, sin variables compartidas globales.

```
-- BIEN: función pura de saturación
function sat_add(a, b : signed; maxv : signed) return signed is
    variable r : signed(a'length-1 downto 0);
begin
    r := a + b;
    if r > maxv then r := maxv; end if;
    return r;
end function;
```

G10. Asignar valor antes de usarlo — Categoría: Warning

Inicializa variables en procesos y da valores por defecto a señales en combinacional para evitar latches.

```
-- MAL: variable sin inicializar
process(sel, a, b) variable yv : std_logic; begin
    if sel='0' then yv := a; end if;
```

```

    y <= yv; -- yv podría no haberse asignado
end process;
-- BIEN: valor por defecto
process(sel, a, b) variable yv : std_logic := '0'; begin
    yv := '0';
    if sel='0' then yv := a; else yv := b; end if;
    y <= yv;
end process;

```

G11. Entradas no conectadas (prohibido en síntesis) — Categoría: Error

Todas las entradas deben conectarse; si no se usan, átelas a un valor conocido o a un pin externo.

```

-- BIEN: atar entradas no usadas
inst: entity work.filtro
port map (
    i_clk => i_clk,
    i_rstn => i_rstn,
    i_en => '0', -- deshabilitado
    i_din => (others=>'0'),
    o_dout => open
);

```

G12. Salidas no conectadas: marcar como 'open' — Categoría: Nota

Si una salida no se usa, declárala como 'open' en el 'port map' para que quede explícito.

```

-- BIEN
uart_i: entity work.uart_rx
port map (
    i_clk => i_clk,
    i_rstn => i_rstn,
    i_rxd => i_rxd,
    o_data => open, -- salida no utilizada
    o_vld => open
);

```

G13. Declarar objetos antes de usarlos — Categoría: Error

Señales, constantes y tipos deben declararse al inicio de la arquitectura/paquete antes de emplearse.

```

-- BIEN
architecture rtl of top is
    constant C_N : integer := 4;
    signal cnt : unsigned(2 downto 0);
begin
    -- uso de C_N y cnt aquí
end architecture;

```

G14. Eliminar declaraciones no utilizadas — Categoría: Warning

Mantén el código limpio: señales/constantes sin uso generan warnings y confunden.

```
-- MAL
signal debug_sig : std_logic; -- nunca usada
-- BIEN
-- eliminarla o, si es temporal, protegerla con generate/ifdef del
entorno
```

G15. Evitar 'record' en puertos de entidad (especialmente top) — Categoría: Warning

La conexión entre entidades debe usar tipos simples (std_logic/_vector).

Empaqueta/desempaqueta records dentro del módulo.

```
-- MAL (top-level)
type axi_t is record
  valid : std_logic;
  data  : std_logic_vector(31 downto 0);
end record;
entity top is
  port ( m_axi : in axi_t ); -- poco portable
end entity;
-- BIEN
entity top is
  port (
    i_valid : in  std_logic;
    i_data  : in  std_logic_vector(31 downto 0)
  );
end entity;

-- Empaquetado interno si se desea
signal m_axi : axi_t;
m_axi.valid <= i_valid;
m_axi.data  <= i_data;
```

Tratamiento de la Señal de Reloj y Reglas RTL

Errores en Tratamiento de la Señal de Reloj

Objetivo: controlar las señales de reloj ante potenciales errores en diseños con múltiples dominios y transiciones asíncronas.

TR01. Analizar múltiples relojes asíncronos — Categoría: Error

Si existen varios relojes no relacionados (o generados internamente), define sus restricciones de temporización, identifica cruces de dominio y coloca resincronizadores/colas apropiadas. Nunca asumas que dos osciladores están alineados.

TR02. Evitar usar ambos flancos del mismo reloj — Categoría: Error

Diseñar lógica en flanco de subida *y* de bajada complica el análisis de temporización y crea restricciones inconsistentes.

```
-- MAL: lógica repartida en ambos flancos
process(clk) begin if rising_edge(clk) then a <= d; end if; end
process;
process(clk) begin if falling_edge(clk) then b <= a; end if; end
process;
-- BIEN: un solo flanco, insertar registros si hace falta
process(clk) begin
    if rising_edge(clk) then
        a <= d;
        b <= a; -- latencia extra en el mismo flanco
    end if;
end process;
```

TR03. Cruces entre dos dominios: usar FIFO de doble reloj u otro CDC seguro — Categoría: Warning

Para transferencias de ancho de palabra, usa una FIFO asíncrona (punteros Gray + doble sincronización). El tamaño compensa las diferencias de frecuencia. Para pulsos/flags, usa handshakes o sincronizadores de 2 FF.

```
-- Ejemplo: sincronizador de 2 FF para un flag de A->B
process(clk_b) begin
    if rising_edge(clk_b) then
        flag_b_1 <= flag_a; -- 1er FF
        flag_b <= flag_b_1; -- 2º FF
    end if;
end process;
```

TR04. Usar un bloque dedicado que genere todos los relojes — Categoría: Warning

Genera los relojes derivados con PLL/MMCM/Clocking Wizard a partir de una única fuente. No derives relojes con lógica general ni con divisores por flip-flop salvo que la herramienta lo promueva a recursos de reloj.

```
-- MAL: dividir el reloj con lógica
process(clk) begin
    if rising_edge(clk) then
        clk_div2 <= not clk_div2; -- reloj interno no distribuido por red
    end if;
end process;
```

```
-- BIEN: mantener un solo reloj y usar 'clock enable'
process(clk) begin
    if rising_edge(clk) then
        if ce_div2='1' then q <= d; end if; -- CE a mitad de frecuencia
    end if;
end process;
```

TR05. No reasignar ni puentear la señal de reloj dentro de una entidad — Categoría: Error

Gating del reloj con AND/OR o reasignarlo crea 'glitches' y rompe la red dedicada de reloj. Usa 'clock enable' o recursos de gating específicos del fabricante.

```
-- MAL
clk_g <= clk and enable; -- gating por lógica combinacional

-- BIEN
process(clk) begin
    if rising_edge(clk) then
        if enable='1' then q <= d; end if; -- clock enable
    end if;
end process;
```

TR06. Preferir un único dominio de reloj por diseño — Categoría: Warning

Simplifica CDC y temporización. Si necesitas varios, sepáralos claramente y documenta las fronteras.

TR07. Preferir un único dominio de reloj por módulo — Categoría: Warning

Un módulo con un solo reloj es más reusable y verificable. Crea módulos específicos para cruce de reloj.

TR08. Un solo flanco activo por dominio — Categoría: Error

Dentro de un mismo dominio todos los registros deben captar en el mismo flanco (subida o bajada).

TR09. Sincronizar señales externas y detectar flancos de forma segura — Categoría: Error

Cualquier entrada asíncrona debe pasar por al menos 2 FF de sincronización antes de la lógica. El detector de flanco se hace sobre la señal ya sincronizada.

```
-- Sincronizador + detector de flanco de subida
process(clk) begin
    if rising_edge(clk) then
        ext_sync1 <= ext_in;
        ext_sync2 <= ext_sync1;
    end if;
end process;
rising_pulse <= ext_sync1 and not ext_sync2;
```

TR10. FSMs claras y predecibles — Categoría: Warning

Separa proceso síncrono (registro de estado) y proceso combinacional (próximo estado/salidas). Evita 'latches', cubre todos los estados y documenta el estado de reset.

```
type state_t is (IDLE, RUN);
signal ps, ns : state_t;
-- síncrono
process(clk) begin
    if rising_edge(clk) then
        if rst='1' then ps <= IDLE; else ps <= ns; end if;
    end if;
end process;
-- combinacional
process(all) begin
    ns <= ps;
    case ps is
        when IDLE => if start='1' then ns <= RUN; end if;
        when RUN  => if done='1' then ns <= IDLE; end if;
        when others => ns <= IDLE;
    end case;
end process;
```

Reglas de código RTL

Prácticas recomendadas para el nivel Register Transfer Level.

RTL01. Puertos en std_logic/std_logic_vector

En interfaces de entidad usa std_logic/_vector. Para aritmética interna emplea signed/unsigned con ieee.numeric_std.

```
-- Puertos (top/entidades)
port (
    i_clk  : in  std_logic;
    i_d    : in  std_logic_vector(15 downto 0);
    o_q    : out std_logic_vector(15 downto 0)
);
```

```
-- Interno
use ieee.numeric_std.all;
signal acc : unsigned(15 downto 0);
```

RTL02. Valores por defecto en entradas de entidad (opcionales)

Puedes dar un valor por defecto a entradas para simplificar instanciación; evita hacerlo en componentes antiguos.

```
entity filtro is
    port (
        i_clk  : in  std_logic;
        i_en   : in  std_logic := '1';  -- enable por defecto
```

```

        i_din   : in  std_logic_vector(7 downto 0);
        o_dout  : out std_logic_vector(7 downto 0)
    );
end entity;

```

RTL03. Direcciones de vectores coherentes

Usa 'downto' cuando el vector represente números; 'to' para índices de matrices (dimensión de memoria, LEDs numerados, etc.).

```

signal data : std_logic_vector(15 downto 0); -- número
type mem_t is array (0 to 15) of std_logic_vector(7 downto 0); -- RAM

```

RTL04. Evitar literales mágicos; usar atributos/constantes

Apóyate en 'length', 'high', 'low' y constantes parametrizables para que el código sea portable.

```

-- MAL
y <= (x(7) xor sign) and x(6 downto 0);
x <= x + 5;

-- BIEN
y <= (x(x'high) xor sign) & x(x'high-1 downto 0);
constant X_INCR : natural := 5;
x <= x + X_INCR;

```

RTL05. Preferir atributos sobre objetos (señales/variables), no sobre tipos

Evita depender de atributos de tipo que hacen el código menos local y más rígido.

RTL06. No usar modo BUFFER en puertos

Usa 'out' + señal interna y, si hace falta, convierte el tipo en la asignación al puerto.

```

-- BIEN
signal out_i : unsigned(7 downto 0);
o_out <= std_logic_vector(out_i);

```

RTL07. Evitar INOUT salvo en nivel superior

Los buses bidireccionales/triestado se modelan en el top; dentro usa señales separadas.

RTL08. Triestado solo en el top

Ejemplo de codificación correcta del bus externo con OE.

```

ExtBus <= BusOut when OE='1' else (others => 'Z');

```

RTL09. Entradas opcionales pueden quedar 'open' si tienen valor por defecto

En la instanciación, deja 'open' o no conectes solo si la entidad define un valor por defecto.

RTL10. Instanciación directa de entidades

Evita declarar componentes; usa 'entity work.foo(rtl)'. Ojo: no configurable con 'configuration'.

```
u1: entity work.filtro(rtl)
  port map (...);
```

RTL11. Puertos no usados: usa 'open'

Haz explícito que no se conecta una salida.

```
u_uart: entity work.uart_rx
  port map (
    i_clk=>i_clk, i_rstn=>i_rstn, i_rxd=>i_rxd,
    o_data=>open, o_vld=>open
  );
```

RTL12. Evitar señales globales y variables compartidas

Aíslalas en módulos o paquetes bien definidos; preferir interfaces explícitas por puertos.

RTL13. Minimizar variables en RTL sintetizable

Usa señales para comunicación entre procesos; variables solo para cálculos locales dentro del proceso.

RTL14. Variables en testbench y modelos comportamentales

En TB las variables facilitan tareas/monitores/scoreboards.

RTL15. Un solo dominio de reloj por entidad (salvo módulos CDC)

Crea módulos dedicados para cruce de reloj y resincronización.

RTL16. No cruzar dominios sin resincronizar

Usa sincronizadores de 2 FF, handshakes o FIFOs asíncronas según el caso.

```
-- Sincronizador 2 FF para bit
process(clk_b) begin
  if rising_edge(clk_b) then
    a2b_1 <= a_flag;
    a2b    <= a2b_1;
  end if;
end process;
```

RTL17. Salidas registradas por defecto

Registra las salidas a la frontera de la entidad para evitar loops combinacionales a través de la jerarquía y facilitar timing.

```
-- Registro de salida
process(clk) begin
  if rising_edge(clk) then
```

```
    o_data <= comb_data;  
end if;  
end;
```

Síntesis Segura (SS) y Revisión de Código (RC) en VHDL

Reglas para una síntesis segura

SS01. Evitar implementaciones peligrosas (feed-throughs, retardos intencionados, triestados internos) — Categoría: Warning

Evita lógica puramente combinacional de extremo a extremo sin registros intermedios, el uso de 'after' para temporizar, y drivers triestado internos (solo en top).

```

-- MAL: feed-through combinacional largo y retardo intencionado (no
sintetizable)
y <= (((a and b) or (c and d)) xor e) after 10 ns;

-- BIEN: registrar para acortar caminos y NO usar 'after'
stagen1 <= (a and b) or (c and d);
process(clk) begin
    if rising_edge(clk) then
        stagen1_r <= stagen1;
        y <= stagen1_r xor e;
    end if;
end process;

-- Triestado interno (MAL). Solo en top:
-- MAL: bus_i <= data when en='1' else 'Z';    -- dentro de módulo
-- BIEN (top): ExtBus <= BusOut when OE='1' else (others=>'Z');

```

SS02. CASE bien definido y completo — Categoría: Error

Cubre todos los valores, sin duplicados ni ramas inalcanzables; incluye 'when others' al menos para capturar errores.

```

-- MAL
case state is
    when IDLE => ...
    when RUN  => ...
end case;  -- falta others

-- BIEN
case state is
    when IDLE => ...
    when RUN  => ...
    when others => state <= IDLE;
end case;

```

SS03. Evitar inferencias de latches — Categoría: Error

Dales valor por defecto a las salidas en procesos combinacionales y completa todas las ramas de if/case.

```

-- MAL: genera latch en y
process(a,b,sel) begin
    if sel='1' then
        y <= a;
    end if; -- cuando sel='0' y no se asigna, se retiene -> latch
end process;

-- BIEN: valor por defecto
process(a,b,sel) begin
    y <= b; -- por defecto
    if sel='1' then

```

```

        y <= a;
    end if;
end process;

```

SS04. No usar asignaciones con formas de onda en RTL — Categoría: Error

Las formas de onda (cola de valores 'after ...') no son sintetizables.

```

-- MAL
q <= '1', '0' after 10 ns; -- solo simulación

-- BIEN (usa registros/contadores para temporización)

```

SS05. Una señal, un único driver secuencial — Categoría: Error

No asignes la misma señal en múltiples procesos o concurrentemente en varios sitios.

```

-- MAL: dos procesos escriben q
process(clk) begin if rising_edge(clk) then if set='1' then q <= '1';
end if; end if; end process;
process(clk) begin if rising_edge(clk) then if rst='1' then q <= '0';
end if; end if; end process;

-- BIEN: centraliza en un proceso
process(clk) begin
    if rising_edge(clk) then
        if rst='1' then q <= '0';
        elsif set='1' then q <= '1';
        else q <= q;
        end if;
    end if;
end process;

```

SS06. Constantes con asignación inmediata — Categoría: Warning

Evita constantes diferidas o dependientes de orden de compilación. Decláralas con valor en el mismo lugar.

```

-- MAL (deferred constant en package, puede complicar build)
package p is
    constant C_WIDTH : natural; -- diferida
end package;

-- BIEN
package p is
    constant C_WIDTH : natural := 16;
end package;

```

SS07. No usar relojes como datos — Categoría: Error

Nunca mezcles el reloj en lógica de datos. El reloj debe ir solo a los pines de reloj de FF/recursos dedicados.

```

-- MAL

```

```
dout <= a and clk;  -- mezclar clk con datos
```

```
-- BIEN
process(clk) begin
    if rising_edge(clk) then
        dout <= a and b;
    end if;
end process;
```

SS08. No usar datos como reloj — Categoría: Error

No generes flancos con señales de datos: habrá 'glitches' dependientes de retardos.

```
-- MAL
process(data_sig) begin
    if rising_edge(data_sig) then  -- ;data_sig no es un clock!
        q <= d;
    end if;
end process;
```

```
-- BIEN: sincroniza data_sig a clk y detecta flanco
process(clk) begin
    if rising_edge(clk) then
        d1 <= data_sig; d2 <= d1;
    end if;
end process;
edge <= d1 and not d2;
```

SS09. No usar el reloj como reset (ni viceversa) — Categoría: Error

El reloj y el reset tienen rutas y requisitos distintos. No intercambiar sus roles.

SS10. No poner lógica en la línea de reloj — Categoría: Error

Nada de AND/OR/XOR sobre el reloj. Si necesitas habilitar, usa 'clock enable' o recursos de gating dedicados.

```
-- MAL
clk_g <= clk and en;  -- posible glitch
```

```
-- BIEN
process(clk) begin
    if rising_edge(clk) then
        if en='1' then q <= d; end if;
    end if;
end process;
```

SS11. Evitar generar dominios de reloj internos — Categoría: Warning

Prefiere un único reloj base y 'clock enable'. Si generas otro reloj, usa PLL/MMCM/recursos de reloj.

SS12. No generar resets asíncronos internamente — Categoría: Warning

Los resets asíncronos internos pueden producir pulsos espurios. Genera resets en el top y resincroniza por dominio.

```
-- BIEN: reset asíncrono externo con desaserción síncrona
process(clk, arst_n) begin
    if arst_n='0' then
        rst_sync1 <= '1'; rst_sync2 <= '1';
    elsif rising_edge(clk) then
        rst_sync1 <= '0'; rst_sync2 <= rst_sync1;
    end if;
end process;
rst <= rst_sync2;
```

SS13. No mezclar polaridades de reset — Categoría: Error

La misma señal no puede ser activa alta en unas partes y activa baja en otras.

SS14. Evitar registros sin reset (según política del proyecto) — Categoría: Warning

Si requieres estado conocido al arranque, incluye reset. Documenta excepciones (p.ej., extensas memorias/arrays).

SS15. Desactivar el reset de forma síncrona — Categoría: Error

La aserción puede ser asíncrona, pero la desaserción debe sincronizarse al reloj para evitar metaestabilidad.

SS16. Evitar valores iniciales de señales en RTL — Categoría: Error

No dependas de inicializaciones en la declaración. Usa un esquema de reset reproducible en HW y simulación.

```
-- MAL
signal q : std_logic := '1'; -- power-up dependiente de tecnología

-- BIEN
process(clk) begin
    if rising_edge(clk) then
        if rst='1' then q <= '1'; else q <= d; end if;
    end if;
end process;
```

SS17. Asegurar controlabilidad de todos los registros — Categoría: Error

Todos los FF deben poder ponerse en un estado conocido desde señales primarias (reset/test).

SS18. Eliminar lógica y registros sin uso — Categoría: Error

Registros nunca leídos o lógica no conducida generan área/potenciales bucles. Revísalos y elimínalos.

SS19. Evitar caminos combinacionales excesivamente largos — Categoría: Warning

Añade etapas de pipeline para cumplir temporización.

```
-- Pipeline para camino largo
stage0 <= f0(a,b,c);
process(clk) begin
    if rising_edge(clk) then
        stage1 <= f1(stage0);
        y      <= f2(stage1, d);
    end if;
end process;
```

SS20. Limitar profundidad de anidamiento/bucles — Categoría: Warning

Anidamientos profundos complican timing y legibilidad. Mantén bucles con límites estáticos (no 'while' abiertos).

```
-- BIEN: límites estáticos
for i in 0 to 7 loop
    ...
end loop;
```

SS21. Consistencia en orden/rango de vectores — Categoría: Warning

Unifica 'downto'/'to' y usa 'high/low/length' para independencia del ancho.

SS22. Usar librerías IEEE estándar (std_logic_1164, numeric_std) — Categoría: Error

Evita paquetes no estándar (std_logic_arith, std_logic_unsigned).

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
```

SS23. Evitar variables en RTL salvo para cálculo local controlado — Categoría: Warning

Las variables no dejan traza inter-procesos. Si las usas, limita su alcance y asigna la salida una sola vez por proceso.

```
process(a,b) begin
    -- OK: variable como acumulador local
    variable acc : unsigned(7 downto 0);
    acc := unsigned(a) + unsigned(b);
    y   <= std_logic_vector(acc);
end process;
```

SS24. Evitar comparaciones de igualdad si hay alternativas más eficientes — Categoría: Warning

Comparar a igualdad sobre buses anchos consume área; a menudo un umbral ('<', '>='), rango o detección de 'cero' es suficiente.

```
-- Ejemplo: terminal de contador
```

```
-- MAL: igualdad exacta a un literal ancho
if cnt = to_unsigned(99999, cnt'length) then ... end if;
```

```
-- BIEN: umbral/rango (si el requisito lo permite)
if cnt >= C_MAX then ... end if;
```

SS25. Comparar usando signed/unsigned (no std_logic_vector) — Categoría: Error

std_logic_vector no tiene semántica numérica: convierte a signed/unsigned antes de comparar o sumar.

```
-- MAL
signal a_slv, b_slv : std_logic_vector(15 downto 0);
if a_slv > b_slv then ... end if; -- significado ambiguo
```

```
-- BIEN
signal a_u, b_u : unsigned(15 downto 0);
a_u <= unsigned(a_slv); b_u <= unsigned(b_slv);
if a_u > b_u then ... end if;
```

Facilitar la revisión de un código

Objetivo: producir código simple, legible y portable que el equipo pueda entender y reutilizar.

RC01. Etiquetas en procesos y bloques — Categoría: Nota

Las etiquetas facilitan depuración y búsqueda. Se preservan en síntesis.

```
comb_mux: process(all) begin ... end process;
reg_proc: process(clk) begin ... end process;
```

RC02. No diferenciar solo por mayúsculas/minúsculas — Categoría: Error

VHDL no es case-sensitive; 'data' y 'DATA' son el mismo identificador. Evita confusiones.

RC03. Evitar mismos nombres en espacios distintos — Categoría: Warning

El mismo nombre en distintos módulos complica búsquedas. Prefiere prefijos contextuales.

RC04. Declaraciones de señales claras (una por línea) — Categoría: Nota

Una declaración por línea mejora el diff y los comentarios.

```
signal data_i : std_logic_vector(7 downto 0); -- entrada
signal data_o : std_logic_vector(7 downto 0); -- salida
```

RC05. Asignaciones claras (una por línea) — Categoría: Nota

Evita empaquetar varias operaciones en una sola línea.

RC06. Indentación y estructura consistentes — Categoría: Nota

Usa dos o cuatro espacios; evita mezclar estilos.

RC07. No usar tabuladores — Categoría: Warning

Distintas herramientas renderizan tabs de forma desigual. Usa espacios.

RC08. Evitar archivos gigantes — Categoría: Warning

Divide por responsabilidad/jerarquía; mejora portabilidad y compilación incremental.

RC09. Nombres significativos según comportamiento — Categoría: Warning

Prefiere 'byte_cnt' antes que 'i'/'n'.

RC10. Cabeceras de archivo coherentes — Categoría: Warning

Incluye autor, fecha, versión, descripción y trazabilidad.

```
--! @brief  UART RX (8N1)
--! @author Equipo HW
--! @version 1.2
--! @date 2025-08-17
```

RC11. Contenido con suficiente valor en cada archivo — Categoría: Warning

Evita archivos triviales que no aportan estructura.

RC12. Comentarios suficientes y útiles — Categoría: Nota

Explican el 'por qué', no lo obvio.

RC13. Convenciones internas de nombres — Categoría: Nota

Define reglas para resets, clocks, enables, activos bajos, etc.

RC14. Llamar 'clk' al reloj — Categoría: Nota

Estandariza: 'clk', 'clk_100m', etc.

RC15. Llamar 'rst' o 'rst_n' al reset — Categoría: Nota

Aclara polaridad y facilita búsquedas.

RC16. Extensión .vhd (o .vhdl) para VHDL — Categoría: Nota

Facilita tooling y búsquedas.

RC17. Nombres de fichero significativos — Categoría: Nota

El nombre debe reflejar el contenido/entidad principal.

RC18. Entidad y fichero homónimos — Categoría: Warning

El fichero debe llamarse como la entidad principal.

RC19. Una entidad por fichero — Categoría: Warning

Encapsula responsabilidades y simplifica navegación.

RC20. Una arquitectura por fichero — Categoría: Warning

Evita confusión; si hay varias, usa archivos separados o comentarios claros.

RC21. Instanciación por nombre, no posicional — Categoría: Error

Reduce errores al añadir/reordenar puertos.

```
-- BIEN
u_uart: entity work.uart_rx
  port map (
    i_clk  => i_clk,
    i_rstn => i_rstn,
    i_rxd  => i_rxd,
    o_data => rx_data,
    o_vld  => rx_vld
  );
```

RC22. Priorizar puertos especiales primero — Categoría: Nota

Declara reloj/reset al inicio del 'port'.

RC23. Simulaciones con duración finita — Categoría: Error

Todo testbench debe terminar (timeout/criterio de fin).

```
-- Testbench: finalización
wait until done='1' for 1 ms;
assert done='1' report "Timeout" severity failure;
```

RC24. Usar procedures/functions en testbench — Categoría: Error

Refactoriza estímulos y checkers; mejora reutilización.

```
procedure drive_byte(signal clk: std_logic; signal rxd: out std_logic;
b: std_logic_vector) is
begin
  -- ...
end procedure;
```

RC25. Usar 'wait' y 'after' solo en testbench — Categoría: Error

Temporiza estímulos con 'wait'/'after' en TB; nunca en RTL.

```
-- Testbench
rxd <= '0', '1' after 104 ns;  -- sólo simulación
```

Bibliografía

[1] Lange M. et al. "Best Practice VHDL Coding Standard for D0-254 Programs.(rev 1a)"

[2] "Design and VHDL Handbook for VLSI and development" CNES